Hardware Description Languages

Viorela Ila

Hardware Description Languages

- Describe hardware at varying levels of abstraction
- Structural description
  - Textual replacement for schematic
  - Hierarchical composition of modules from primitives
- Behavioral/functional description
  - Describe what module does, not how
  - Synthesis generates circuit for module
- Simulation semantics

Hardware Description Languages

- Abel (circa 1983) - developed by Data-I/O
  - Targeted to programmable logic devices
  - Not good for much more than state machines
- ISP (circa 1977) - research project at CMU
  - Simulation, but no synthesis
- **Verilog** (circa 1985) - developed by Gateway (absorbed by Cadence)
  - Similar to Pascal and C
  - Delays is only interaction with simulator
  - Fairly efficient and easy to write
  - IEEE standard
- **VHDL** (circa 1987) - DoD sponsored standard
  - Similar to Ada (emphasis on re-use and maintainability)
  - Simulation semantics visible
  - Very general but verbose
  - IEEE standard

Verilog vs. VHDL

```
module fulladd4(
    input wire [3:0] a,
    input wire [3:0] b,
    input wire cin,
    output wire [3:0] sum,
    output wire cout
);
    assign {cout, sum} = a + b + cin;
endmodule
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity fulladd4 is
    Port (
        a : in STD_LOGIC_VECTOR (3 downto 0);
        b : in STD_LOGIC_VECTOR (3 downto 0);
        cin : in STD_LOGIC;
        sum : out STD_LOGIC_VECTOR (3 downto 0);
        cout : out STD_LOGIC
    );
end fulladd4;

architecture Behavioral of fulladd4 is
begin
    process(a, b, cin)
    variable temp: STD_LOGIC_VECTOR(4 downto 0);
    begin
        temp := ('0' & a) + ('0' & b) + cin;
        sum <= temp(3 downto 0);
        cout <= temp(4);
    end process;
end Behavioral;
```

Introduction to Verilog (Verifying Logic)

- Verilog, standardized as IEEE 1364 in 1995 (**Verilog-95**) , is a hardware description language (HDL) used to model electronic systems.
- Extensions to Verilog-95 became IEEE Standard 1364-2001 known as **Verilog-2001**, another update occurred in 2005. In 2009, the SystemVerilog and Verilog were merged into SystemVerilog 2009 which is now IEEE Standard 1800-2009.
- It is commonly used in the design and verification of digital circuits at the **register transfer level** of abstraction.
- In Verilog, a design consists of **modules**.
- A simple AND gate in Verilog would look something like the following:

```
// AND gate                          <-comment
module andgate (x1, x2, f);   <- module name + sensitivity list
    input x1, x2;
    output f;
    assign f = x1 & x2;        <- assignment statement
endmodule
```

Three Modeling Styles in Verilog

There are three modelling styles in Verilog:

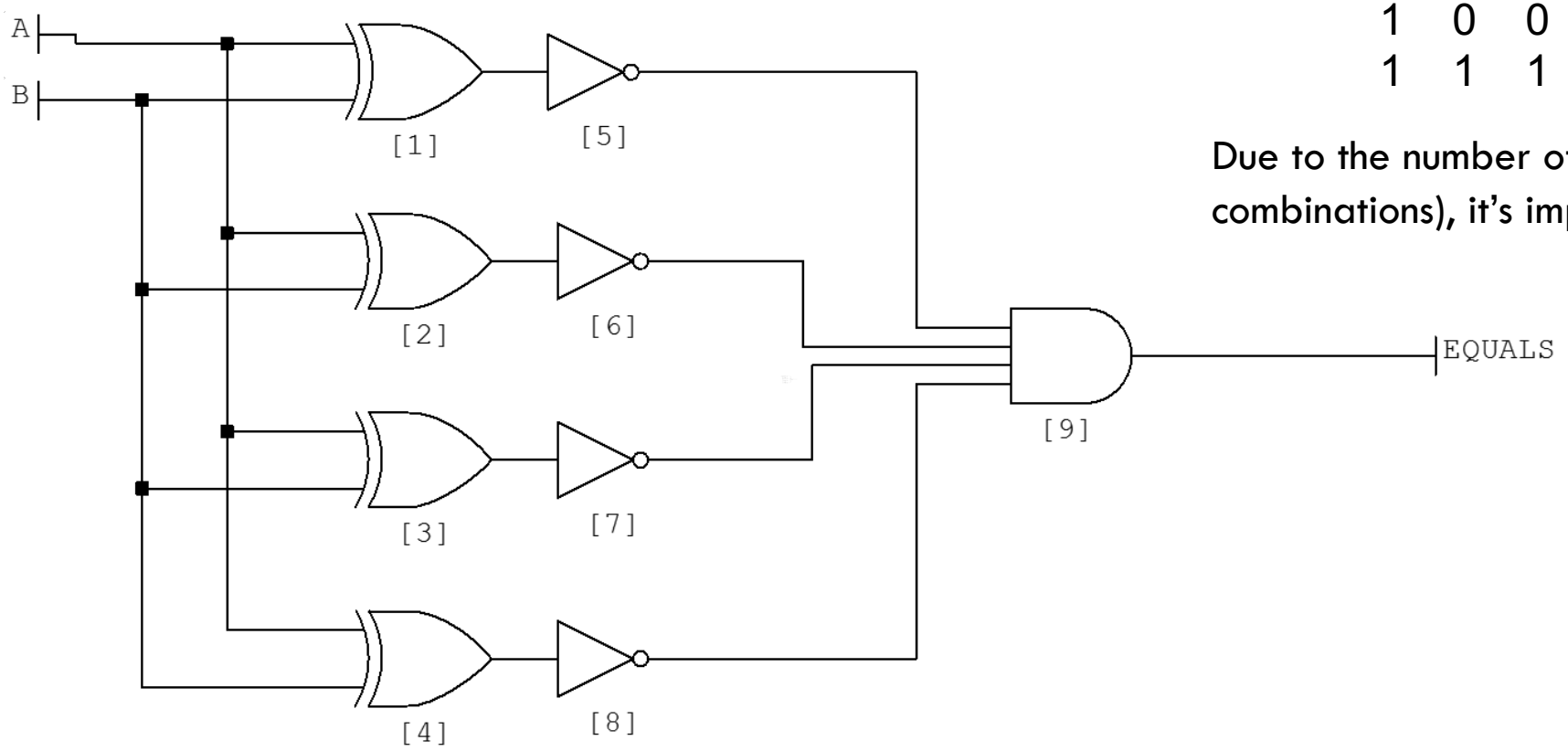
Structural modeling (Gate-level): Use predefined or user-defined primitive gates.

Dataflow modeling: Use assignment statements (**assign**)

Behavioral modeling: Use procedural assignment statements (**always**)

4-bit Comparator Example

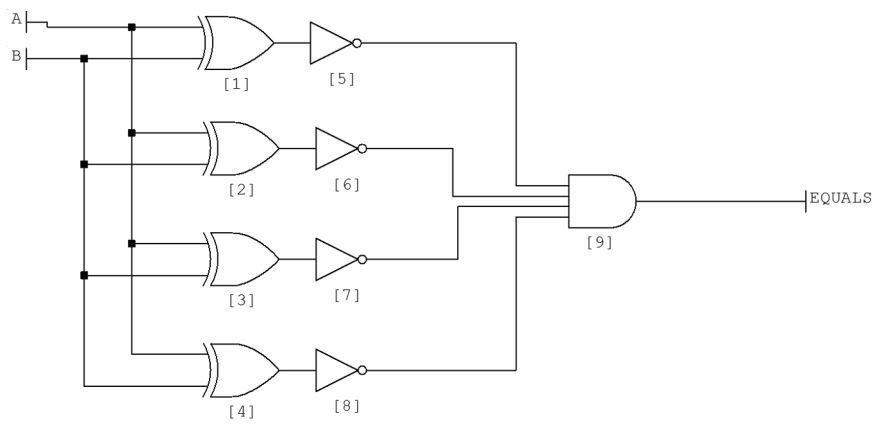
A3	A2	A1	A0	B3	B2	B1	B0	A = B
0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	0	1
0	1	0	1	0	1	0	0	0
1	0	0	0	0	0	1	1	0
1	0	0	1	1	0	0	1	1
1	1	1	1	1	1	1	1	1



Due to the number of possible combinations (2^8 = 256 combinations), it's impractical to list all!

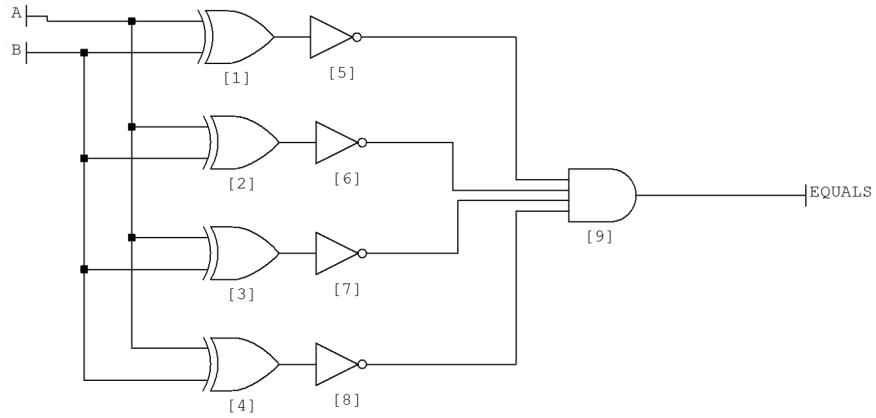
Structural 4-bit Comparator

```
module comparator(input[3:0] a,b, output out);
    wire n0, n1, n2, n3;
    xnor xnor0( n0, a[0], b[0] );
    xnor xnor1( n1, a[1], b[1] );
    xnor xnor2( n2, a[2], b[2] );
    xnor xnor3( n3, a[3], b[3] );
    and and0( out, n0, n1,n2, n3 );
endmodule
```



Dataflow 4-bit Comparator

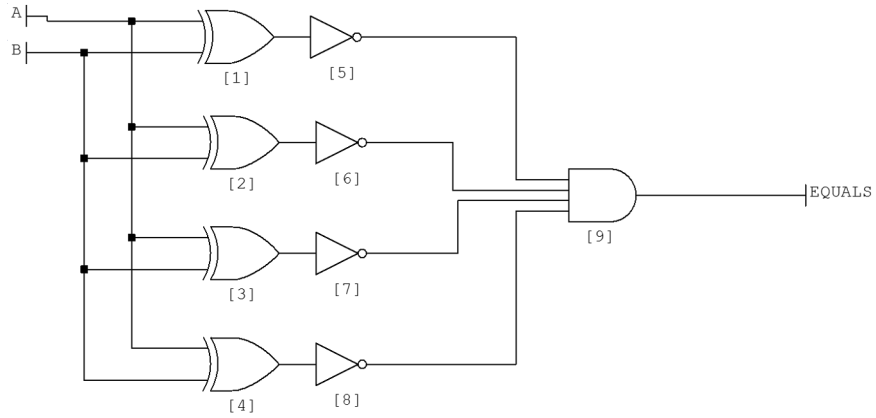
```
module COMPARATOR (A, B, Y); //Verilog 1995
    input [3:0] A, B,
    output Y;
    assign Y = & ( (A~^B));
endmodule
```



The way to understand the **concurrent assign statement** is that vectors A and B are XNOR 1-bit at a time: The **caret** symbol represents the XOR operation in Verilog, and the **tilde** represents an inverter. The four bits are then AND together as represented by the **ampersand** in front of the parenthesis expression there, and then that gets assigned to the output y.

Behavioral 4-bit Comparator

```
module COMPARATOR (input [3:0] A, B; output Y);
    integer N;
    reg Y;
    always @(A or B)
        begin: COMPARE
            Y = 0;
            if (A == B)
                Y = 1;
        end
endmodule
```



In the behavioral model, a special block of code known as the **always** block is used where the statements inside are sequential and not concurrent as they are outside of an always block.
Whenever **there is a change to A or B**, the two vectors are compared, and if equal, the output is changed to a one, otherwise it remains zero. The synthesizer will again create exactly the same comparator circuit based on this description.

Sequential vs. Concurrent Statements

Sequential Statements:

- Execute in order (one after another)
- Found inside always blocks
- Use procedural assignments (= or <=)
- Support control structures like if/else, case, loops

```
always @(posedge clk)
begin if (reset) count <= 0;
      else count <= count + 1;
end
```

Concurrent Statements:

- Execute simultaneously
- Include assign statements
- Use continuous assignments
- Order doesn't matter

```
assign sum = a + b; // Both execute
assign carry = a & b; // simultaneously
```

Introduction to Verilog (Verifying Logic)

Basic Concepts and Syntax

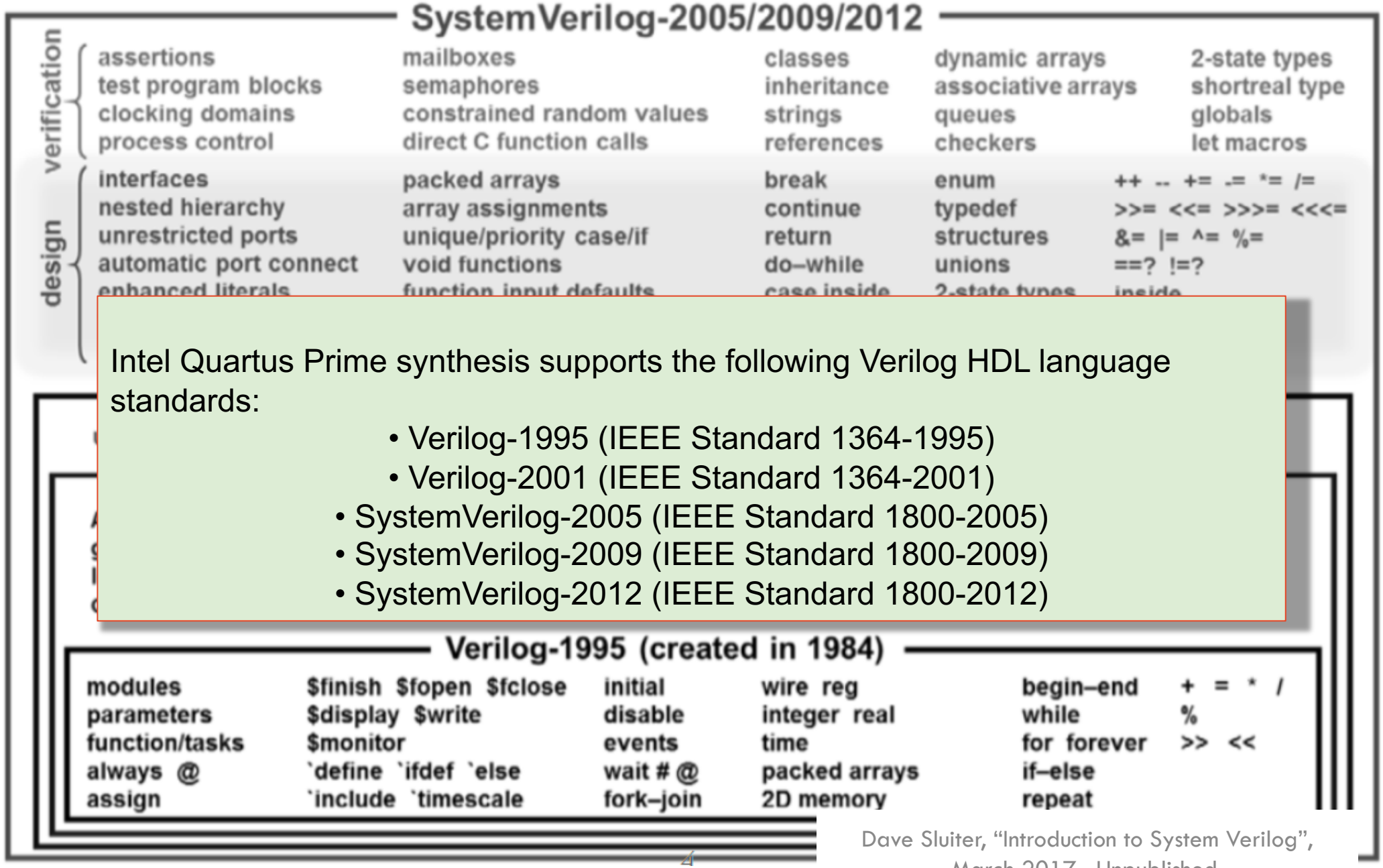
- **Data Types:** Understanding the different data types such as `reg`, `wire`, `integer`, and their uses.
- **Operators:** Familiarity with Verilog operators like arithmetic, logical, relational, bitwise, and reduction operators.
- **Modules and Ports:** Knowing how to define modules, the building blocks of Verilog code, and how to declare and connect ports (input, output, inout).

Verilog Syntax - Overview

The advantage is that many syntax rules are much like in C language:

- **Whitespace** is ignored
- **Comments** are either //… or /* … */
- **Identifiers** are words for variables, module names, etc. and can begin with _ or a letter and can include letters, digits, and underscores and are CASE SENSITIVE!
- **Keywords** can NOT be used as identifiers, they are:
  - **assign,**
  - **case,**
  - **while,**
  - **wire, reg,**
  - **and, or, nand, …**
  - **module**
  - **always**

  Etc.



Intel Quartus Prime synthesis supports the following Verilog HDL language standards:

- Verilog-1995 (IEEE Standard 1364-1995)
- Verilog-2001 (IEEE Standard 1364-2001)
- SystemVerilog-2005 (IEEE Standard 1800-2005)
- SystemVerilog-2009 (IEEE Standard 1800-2009)
- SystemVerilog-2012 (IEEE Standard 1800-2012)

Dave Sluiter, "Introduction to System Verilog", March 2017. Unpublished.

Verilog Numbers

- Numbers can be assigned to variables in two ways they can be **sized** or **unsized.**
- If you don't specify a size or format for a number, they will default to a **32-bit decimal number.**
- Number literals are expressed this way:

  3'b001, a 3-bit number
  5'd30, (=5'b11110), a 5-bit decimal number
  16'h5ED4, (=16'd24276)(=16b0101111011010100), a 16-bit hexadecimal number

- In general: **size_in_bits'baseNumber**

- Underscores can be added for improved readability as in 32'h1234_5678 or 16b0101_1110_1101_0100

Verilog Values

Verilog consists of only four basic values. Almost all Verilog data types store all these values:

0 (logic zero, or false condition)

1 (logic one, or true condition)

Only for physical data types and limited use for synthesis.

x (unknown logic value or contention).

z (high impedance state, floating).

```
assign wire1 = 1'b1;  // Driver A says "1"
assign wire1 = 1'b0;  // Driver B says "0"
// Result: wire1 = X (conflict!)

module tristate_buffer(
    input wire data_in,
    input wire enable,
    output wire data_out );
    assign data_out = enable ? data_in : 1'bZ;
endmodule
```

Verilog Values

Constants in Verilog are expressed in the following format:
size_in_bits'base*Value*
size_in_bits — Expressed in decimal integer. Optional, default is inferred from value.
'*base* — Binary(b), octal(o), decimal(d), or hexadecimal(h). Optional, default is decimal.
value — Any value can be binary, octal, decimal or hexadecimal.

```
4'b1011 // 4-bit binary of value 1011
234 // 32-bit 3-digit decimal of value 234
8'h5a // 8-bit  hexadecimal of value 5A
9'o671 // 9-bit octal of value 671
4'b1x0z // 4-bit binary. 2nd MSB is unknown. LSB is Hi-Z.
3.14 // Floating point
1.28e5 // Scientific notation
```

Verilog Parameters

- Parameters can be used to associate a value to a symbolic name. This is done using the keyword **parameter** thus you can give information like size and settings symbolic names to make the code more readable and meaningful.
- Must resolve to a **constant** at compile time
- Can be overwritten at compile time
- Keywords:

  `parameter`

  `localparam` // same as `parameter` but cannot be overwritten

  Eg.

```
parameter size = 8;
localparameter outsize = 16
reg[size-1:0]data,datab;
reg[outsize-1:0]out
```

- Can be included in the module declaration (Verilog-2001)

Verilog Data Types

Significant Data types available in Verilog include:

wire // A wire represents a signal on a wire that's being driven continuously by a driver
reg, trireg // designed for variables that have some kind of a storage element
integer
real
time // a unique type of data type within Verilog, which is used to represent time
parameter //allows you to parametrise the design
event // is something that happens at a particular point in time that can be used to launch other activities and so on and simulation

Physical data types (**wire, reg, trireg**) can be declared as vector/bus (multiple bit widths).
An **array** is a chunk of consecutive values of the same type.
Data types **reg**, **integer** and **time** can be declared as an array.

```
reg [7:0] memory [0:255][0:15]; // 256x16 array of 8-bit words
```

Verilog Data Types

Net Types

Connections between hardware elements and are typically used to model the wiring in a digital circuit.

- **wire**: Represents a physical wire. The default net type.
- **tri**: Similar to wire, but used for tri-state logic.
- **wor**: Wired OR. Multiple drivers are OR-ed together.
- **wand**: Wired AND. Multiple drivers are AND-ed together.
- **tri0**: Tri-state logic with a default logic value of 0 when not driven.
- **tri1**: Tri-state logic with a default logic value of 1 when not driven.
- **supply0**: Constant logic 0.
- **supply1**: Constant logic 1.
- **trireg**: A tri-state register that retains its last value when undriven (capacitive storage).

Verilog Data Types

Variable Types

Variables are used to store data and can be assigned values within procedural blocks.

- **reg**: Used to store binary data, can be used in procedural blocks (always, initial).
- **integer**: Used for integer values.
- **real**: Used for real number values.
- **realtime**: Similar to real, but used for time-related calculations.
- **time**: Used for storing simulation time values.
- **logic**: Introduced in SystemVerilog, similar to reg but can be used in more contexts.
- **bit**: A two-state data type (0 or 1), introduced in SystemVerilog.
- **byte**: An 8-bit data type.
- **shortint**: A 16-bit signed integer.
- **int**: A 32-bit signed integer.
- **longint**: A 64-bit signed integer.
- **shortreal**: A 32-bit floating-point number.
- **string**: A dynamic array of bytes.

Verilog Data Types

Constant Types

Constants are used to define fixed values.

- **parameter**: A constant value that can be used throughout a module.
- **localparam**: Similar to parameter, but local to the module and cannot be overridden.

Physical Types

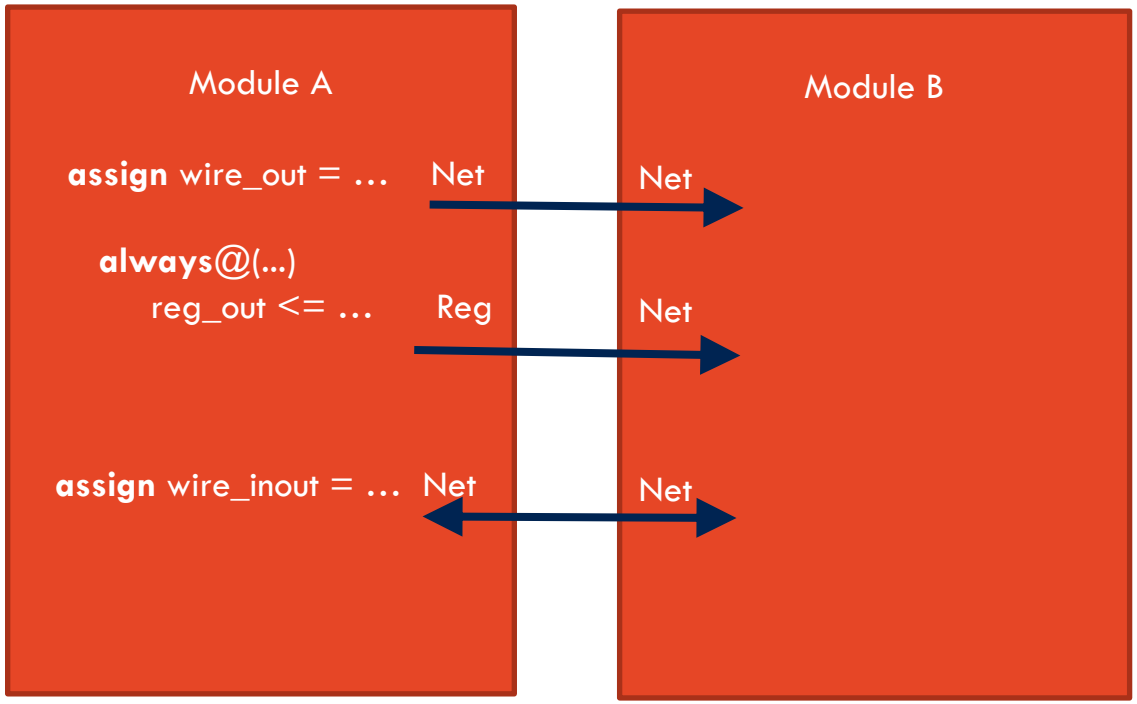
Physical types are used to model physical quantities like time and current.

- **time**: Used for simulation time values.
- **trireg**: As mentioned earlier, it retains its value when undriven, modeling capacitive storage.

Ports

Port Rules

- Ports define the interface of a Verilog module to the outside world.
- **Input**s must be Nets (wire, etc.)
- **Output**s can be Nets or Registers
- **Inout** must be Nets (bidirectional)
- The Left Hand Side (LHS) of **procedural assignment**s must be of a **Register** type.
- For **continuous assignment**s outside of procedural blocks, LHS must be **Nets** (wires).



Arithmetic Operators

Expression	Operator used	Operation performed
a + b	+	Add
a – b	–	Subtract
a * b	*	Multiply
a / b	/	Divide
a % b	%	Modulus(the remainder is given as result)
a ** b	**	Power(exponent)

```
//unary arithmetics
-4 // negative number
+5 // positive number

//binary arithmetics
5+2 = 7 // addition
6-4 = 2 // subtraction
2*3 = 6 // multiplication
20/5 = 4 // division
-10/5 // correct result
-32'd10/5 // incorrect result
```

- The operators treat input vectors as a whole number
- Result is unknown if any of the operands are Z or X
- **Carry bit(s)** are handled automatically if the result is wider than the operands, **it is lost** if the result is the same size as the operand
- Note: It is advisable to write negative numbers in real or integer format. Verilog converts negative numbers into 2's complement internally, writing in <size>'<base><number> format will yield incorrect result.

Example of 8-bit Arithmetics

```
module arithmetic_operations;
    reg[7:0] data1;
    reg[7:0] data2;
    initial
       begin
           data1 = 45;
           data2 = 9;
           $display ("add(data1,data2) = %d", data1 + data2);
           $display ("subtract(data1,data2) = %d", data1 - data2);
           $display ("multiply(data1,data2) = %d", data1 * data2);
           $display ("divide(data1,data2) = %d", data1 / data2);
           $display ("modulus(data1,data2) = %d", data1 % data2
           $display ("power(data2,2) = %d", data2 ** 2);
       end
endmodule
```

Logic Operators

Expression	Operator	Operation	Description
A && B	&&	Logical – AND	The result will be 1 (true) if A and B are true
A || B	||	Logical – OR	The result will be true if either A or B is true
!A	!	Logical negation	Will convert to zero if A is non-zero or 1 A is zero or false value

- Used to evaluate single expressions or compare multiple expressions
  - Each operand is considered a single expression
  - Expressions with a zero value are viewed as false (0)
  - Expressions with a non-zero value are viewed as true (1)
- Returns a **1-bit scalar** value of Boolean true(1)/false(0)
- X or Z are both considered unknown in operands and result is always unknown

Logic Operators

```
module logical_operators ();

  reg r_A = 1'b1;
  reg r_B = 1'b0;

  // Initial statement is not synthesizable (test code only)
  initial
    begin
      #10;
      if (r_A || r_B)
        $display("Either r_A or r_B is 1");
      else
        $display("Neither r_A or r_B is 1");

      if (r_A && r_B)
        $display("Both r_A and r_B are 1");
      else
        $display("r_A and r_B are not both 1");

      if (!r_A)
        $display("r_A is not 1");
      else
        $display("r_A is 1");
    end
endmodule // logical_operators
```

Bit-wise Operators

Expression	Operator	Description
A & B	&	AND each bit
A | B	|	OR each bit
A ^ B	^	XOR each bit
~A	~	NOT invert each bit
A ^~ B	^~ or ~^	XNOR each bit

```
X=4'b1010;
Y=4'b1101;
Z=4'b10x1;

Then:
~X      is 4'b0101
X & Y   is 4'b1000
X | Y   is 4'b1111
X ^ Y   is 4'b0111
X ^~ Y  is 4'b1000
X & Z   is 4'b10x0
```

- Bit-wise operators operate on each bit of the operand
- **Result is the size of the larger operand**
- X or Z are both considered unknown in operands, but the result might be known value
- Operands are left-extended if size are different

Reduction Operators

Expression	Operator	Description
&A	&	Performs bitwise AND operation on A
|A	|	Performs bitwise OR operation on A
^A	^	Performs bitwise XOR operation on A

```
&(4'b1011) = 1 & 0 & 1 & 1 = 0
// reduction and of 1011
```

- Unlike logical and bitwise logical operators, the Reduction operator is a unary operator.
- This operator is useful for converting a multi-bit vector into a single bit scalar value.
- It performs bit by bit logical operation on the vector operand and returns a Boolean value.

Comparison

Logical	Bitwise logical	Reduction
Binary operator (except negation)	Binary operator (except negation)	Unary operator
Returns a 1-bit Boolean value	The return value is of the same size as the operands	Returns a 1-bit Boolean value
Evaluate the logical values of the operands and then perform a logical operation.	Performs a logical operation on each bit of operand with the corresponding bit of other operands	Evaluate each bit of vector operand and convert it into a scalar by performing a logical operation.

Relational operators

Expression	Operator	Description
a>b	(greater than)	Returns 1 if a is greater than b
a<b	< (less than)	Returns 1 if a is less than b
a<=b	<= (less than or equal to)	Returns 1 if a is either less than or equal to b
a>=b	>= (greater than or equal to)	Returns 1 if a is either greater than or equal to b

```
reg[7:0] A;
reg[7:0] B;
A = 3;
B = 4;

A <= B; // returns 1
A > B; // returns 0
```

- Used to compare values
- Return a 1 bit scalar value of Boolean true (1) / false (0)
- X or Z are both considered unknown in operands and result is always unknown

Equality operators

Expression	Operator	Description
A = = B	= =	A equal to B, result is unknown if A or B has z(high impedance) or x(unknown)
A != B	! =	A not equal to B, result is unknown if A or A has z(high impedance) or x(unknown)
A = = = B	= = =	A equal to B including x(unknown) and z(high impedance)
A ! = = B	! = =	A not equal to B including x(unknown) and z(high impedance)

```
reg[7:0] A;
reg[7:0] B;
A = 4'bx10z;
B = 4'b010z;

A == B;  // returns x
A === B; // returns 0
```

- Logical Equality (==,!==): In this case, if one of the operand bits has an x(unknown) or z(high impedance), the resultant will be x.
- Case Equality (===,!===): Here, the x(unknown) and z(high impedance) in operand bits are included during the comparison. So the result will be either true (1) or false (0).

Shift operators

Operator	Description
>>	Right shift
<<	Left Shift
>>>	Arithmetic Right Shift
<<<	Arithmetic Left shift

```
module shift_operators;
  reg [3:0] var1 = 4'b1000;
  reg signed [3:0] var2 = 4'b1000 ;
  initial
  begin

    //left shift
    $display("%b", var1 <<  1);
    $display("%b", $signed(var1) <<< 1); // Cast as signed
    $display("%b", var2 <<< 1); // Declared as signed type

    // Right Shift
    $display("%b", var1 >>  2);
    $display("%b", $signed(var1) >>> 2); // Cast as signed
    $display("%b", var2 >>> 2) ;  // Declared as signed type
  end
endmodule

Result:
0000 0000 0000 0010 1110 1110
```

- **Logical shift:** they shift the input and pad with zeros. For example, shift 4'b1000 right twice will result in 4'b0010.
- **Arithmetic shift:** they preserve the sign of MSB of our variable we need to shift. For example, arithmetic shift 4'b1000 to the right twice will be 4'b1110.

Concatenation Operators

- Concatenation operators are used to join different bits of data into one. Concatenations are expressed using the brace characters **{ }**, with commas separating the expressions within.

```
module concatenation_operator;
  reg A;
  reg[1:0] B, C;
  reg[2:0] D;

  initial
  begin

    A = 1'b1;
    B = 2'b00;
    C = 2'b10;
    D = 3'b110;

    $display ("concatenation(B,C) = %b", {B,C}); // two 2 bits joined to form 4 bit number
    $display ("concatenation(A,B,C,D,3'b001) = %b", {A,B,C,D,3'b001});
    $display ("concatenation(A,B[0],C[1]) = %b", {A,B[0],C[1]});

  end
endmodule
```

Result:

```
concatenation(B,C) = 0010
concatenation(A,B,C,D,3'b001) = 10010110001
concatenation(A,B[0],C[1]) = 101
```

Conditional Operator

The conditional operator selects an expression for evaluation depending on the value of the condition:

condition?true_expression:false_expression

If the condition is evaluated as false (or zero value), then false_expression is evaluated and used as a result of an entire expression.

```
assign out = enable?data1:data2;
```

Replication Operator

- The replication operator is used to replicate a group of bits n times. It takes the format {n{m}}, where n indicates replication multiplier i.e., how many times m should be repeated.
- It is important to note that the replication multiplier must be a constant.

```
module replication_operator;
  reg A;
  reg[1:0] B, C;
  reg[2:0] D;
  wire Y;
  initial
  begin
    A = 1'b1;
    B= 2'b00;
    C = 2'b10;
    D = 3'b110;
    $display ("replicating A = %b", {4{A}});
    $display ("replicating A and B = %b", {{4{A}},{2{B}}});
    $display ("replicating A,B,C = %b", {{4{A}},{2{B}},{C}});
  end
endmodule
```

Result

```
replicating A = 1111
replicating A and B = 11110000
replicating A,B,C = 1111000010
```

Precedence

Precedence	Operators	Operator Symbols
Highest	Unary Multiply Divide, Modulus	+ – ! ~ * / %
	Add Subtract Shift	+ – << >>
	Relational Equality	< <= > >= == !== === !===
	Reduction Logical	& ~& ^ ^~ | ~| && ||
Lowest	Conditional	?:

Assignments

- The fundamental statement in Verilog is the **assignment** statement.

- Verilog there are two ways to make assignments:
  - **continuous** assignment statements and
  - **procedural** assignment statements

- All assignment statements outside of an **always** block are **concurrent** – they happen at the same time, and are not sequential.
- The output of the operation on the right hand side of the = symbol is continuously assigned to the variable on the left hand side, as in

```
assign A = B & C;   // an and gate
```

- The variable on the left hand side must be a **net**, not a **reg** when outside of an always block.

Continuous Assignments

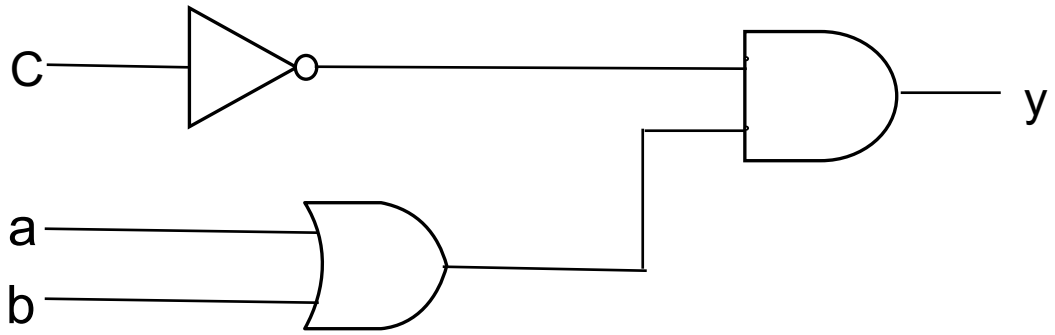
- A continuous assignment is used to represent **combinational logic.**
- Continuous assignments are **always active.**
- Left-hand side must be a **net** data type.
- Right hand side can be a net, reg or functional calls.
- We can have continuous assignments using the keyword **assign**
- Delay values can be assigned to model gate delays

```
// using keyword
wire[15:0] adder_out
assign adder_out = mul_out + out

//delay
assign #5 adder_out = mul_out + out
```

Continuous Assignments

How would this be coded in Verilog?



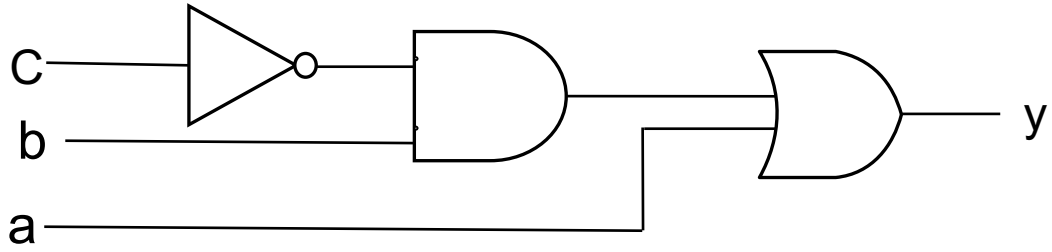
Answer:

```
assign y = (a | b) & ~c;
```

Note that if we leave out the parenthesis:

```
assign y = a | b & ~c;
```

we get:



Because of the default order of evaluation.

Procedural Assignments

- Procedural assignments are made inside procedural assignment blocks.
- There are two types of procedural assignment blocks:
  - **initial** blocks
  - **always** blocks

- **Initial** blocks are typically used to initialize behavioural statements for simulation.
- **Always** blocks are always executing in a **looping** fashion.

- Each **always** and **initial** block represent a separate process.
- The processes all run in parallel but the statements within the process are **sequential.**
- Each process begin at time 0 in simulation.
- **always** and **initial** blocks cannot be nested

Initial Block

- Consists of behavioural statements
- An initial block starts at simulation time 0, executes only once during the simulation.
- Keywords **begin** and **end** must be used if the block contains more than one statement
- Example of use:
  - Initialization
  - Monitoring
  - Any functionality that needs to be turned on just once
- An **initial** block **is not synthesizable** so cannot be converted in hardware

```
initial
      [single statement]
initial begin
      [multiple statements]
end
```

Always Block

- Consists of behavioural statements
- Multiple blocks execute concurrently starting at time 0 and **continuously looping.**
- Keywords **begin** and **end** must be used is the block contains more than one statement.
- Behavioural statements inside an **always** block execute sequentially
- Example of use:
  - Modelling a digital circuit
  - Any process or functionality that needs to be executed continuously
- The **always** block is executed at a particular **event**

```
always @ (event)
      [single statement]
always @ (event)
begin
      [multiple statements]
end
```

Types of Procedural Assignments

Procedural **blocking** assignments "="

- executed in the order they are specified in a sequential block
- a second assignment is not started until the preceding one is complete.

Procedural **nonblocking** assignments "<="

- allow scheduling the assignment without blocking the execution of the statements that follow in a sequential block
- assignments which follow each other in the code, are started in parallel.

Blocking vs Nonblocking Assignments

The left hand side of the assignment can be: reg, integer, real or time
The right hand side of the assignment can be of any type of expression or signal

The right hand side of nonblocking assignments is evaluated starting from the completion of the last blocking assignment or if none, the start of the procedure.

```
input wire b;
reg      a,c;
integer  count;
real     value;
time     delay;

always @(*) begin
    a     = b; // blocking
    count = count + 1;
    value = 3.14;
    delay = $time;
    c    <= a; // nonblocking
end
```

Blocking vs Nonblocking Assignments

Blocking: Order matters!

```
always @( posedge clk)
   begin
      Q2=Q1;
      Q1=D; // is a shift register  Q2 = Q1(old), but Q1 = D (new)
   end
```

Vs.

```
always @( posedge clk)
   begin
      Q1=D;
      Q2=Q1; // is a single or parallel FF
   end
```

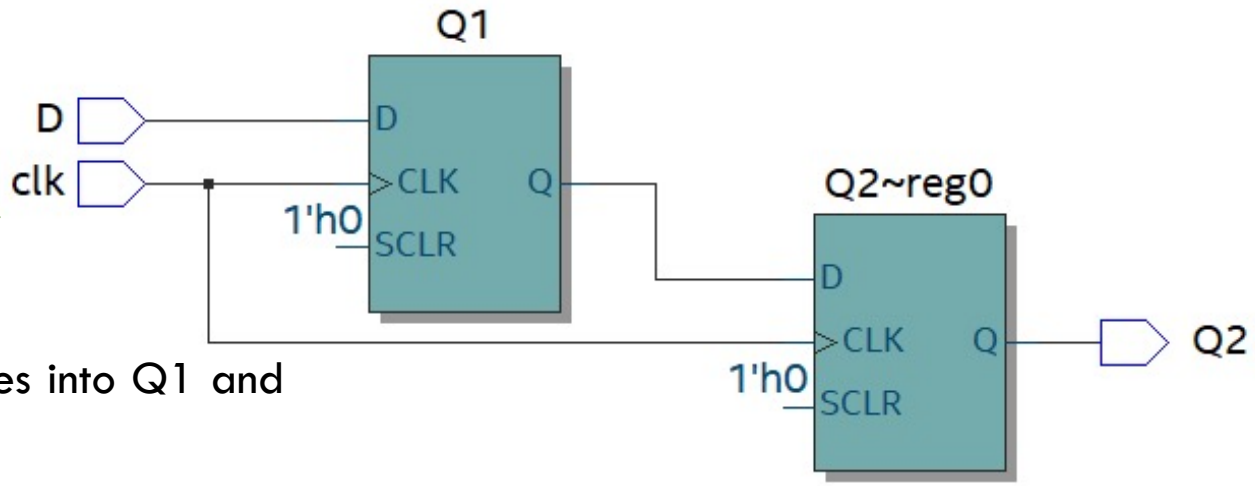
Blocking vs Nonblocking Assignments

Blocking: Order matters!

```
always @( posedge clk)
    begin
        Q2=Q1; Q1=D; // is a shift register
    end
```

The synthesizer creates a **shift register** where you have D goes into Q1 and then Q1 goes into Q2 and so on.

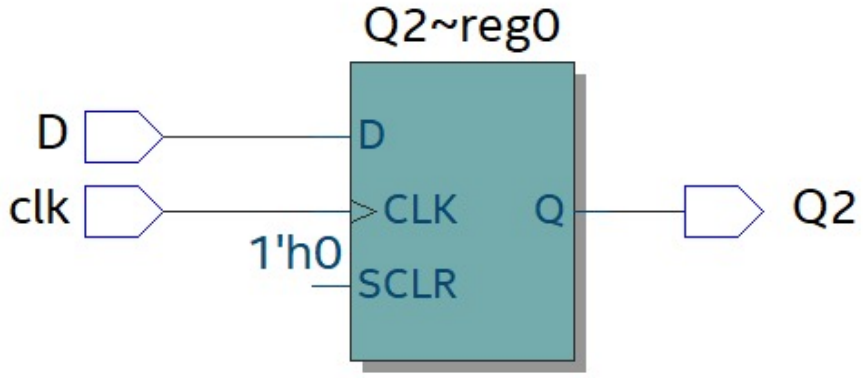
Quartus Synthesizer Output:



Vs.

The synthesizer is going to either generate a **single flip-flop** where D is the input and Q2 is the output, or it might create **parallel flip-flops** where you have both Q1 and Q2 outputs, but D is driving both of those.

```
always @( posedge clk)
    begin
        Q1=D; Q2=Q1; // is a single or parallel FF
    end
```



Blocking vs Nonblocking Assignments

Non-Blocking: Order does not matter!

```
always @( posedge clk)
   begin
      Q2<=Q1; Q1<=D; // PROPER shift register Q2 = Q1(old), Q1 = D
   end
always @( posedge clk)
   begin
      Q1<=D; Q2<=Q1; // shift register exactly the same
   end
always @( posedge clk)
   begin
      Q1<=D; Q2<=D;  // parallel ff
   end
```

When you have **nonblocking** assignments and they follow each other in the code, they're all started in parallel, so they are concurrent.

Verilog Fundamentals

Guidelines "Golden Rules":

- One generally should not mix "<=" or "=" in the same procedure.
- "<=" best mimics what physical flip-flops do;

  use it only for "**always** @ (**posedge** clk ..) type procedures describing sequential circuits.
- "=" best corresponds to what C/C++ code would do;

  use it for combinational procedures, within or outside of always blocks.
- If we combine assignment statements with operators on variables, then we get a wide variety of possible circuit descriptions.

When to Use Blocking Assignments

```
// Simple example: Decoder with intermediate logic
 module decoder_example(
    input wire [1:0] select,
    input wire enable,
    output reg [3:0] output_val
 );
    reg valid_select; // Intermediate variable
    always @(*)
    begin
        // Step 1: Check if select is valid (blocking assignment)
        valid_select = (select != 2'b11);
         // Step 2: Use the updated valid_select value
        if (enable && valid_select)
            output_val = 4'b1 << select; // Decode
        else output_val = 4'b0000; // Default
    end
endmodule
 // WHY blocking (=) is needed: valid_select must be calculated
BEFORE it's used
```

Verilog Structure

Modules

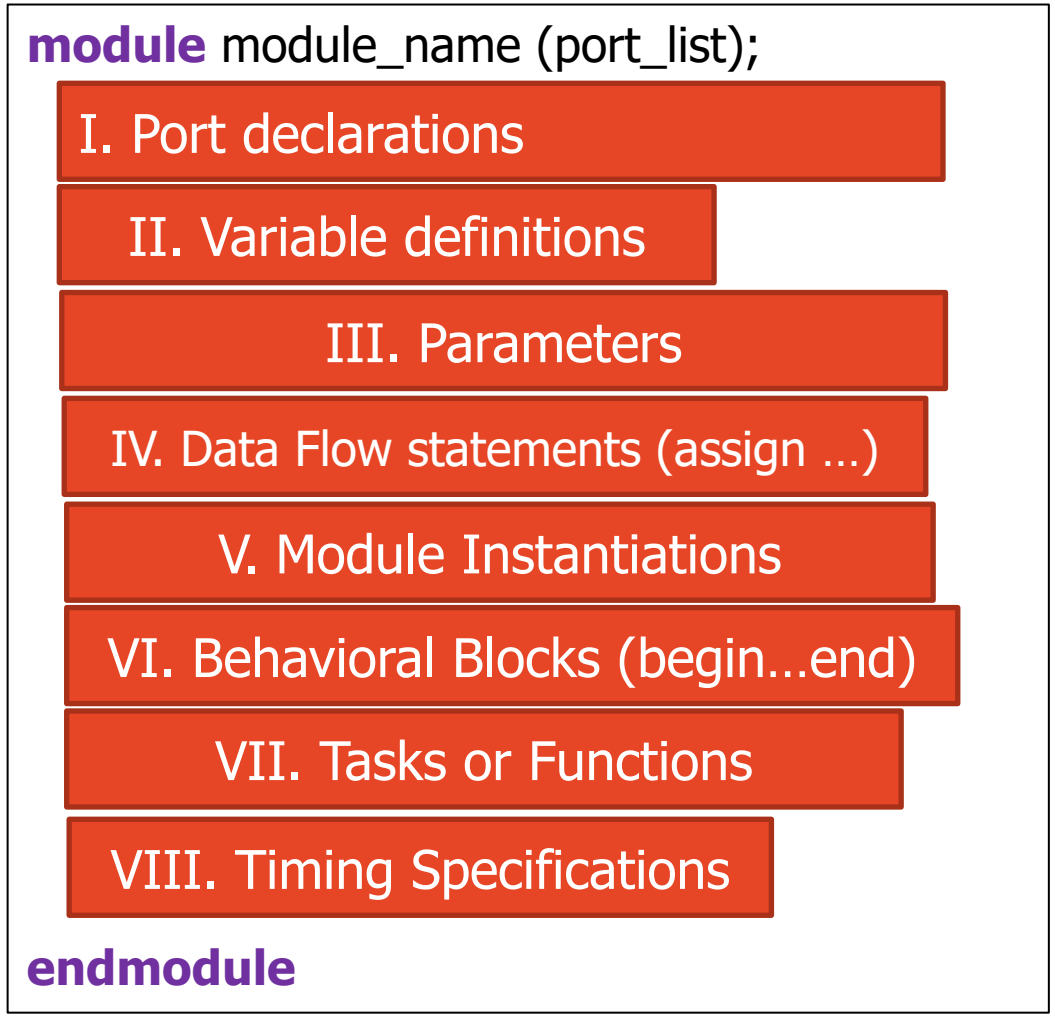
Modules are the basic building blocks in Verilog. A module definition always starts with the keyword module, followed by the **module name, port list, port declarations,** and **parameters.**

This is followed in any order by:

- Variable definitions (local scope)
- Dataflow statements
- Instantiation of lower modules
- Behavioral Blocks
- Tasks or Functions

Followed by the keyword endmodule (no semi-colon).

Verilog Structure



Verilog Structure

Module Instantiation

Modules can be instantiated by using **ordered port lists**, or by **port names** which don't need to follow the module order. For example, here are 2 ways to instantiate a full adder:

```
reg [3:0] A, B;    // top level signals in caps
reg C_IN;
wire [3:0] SUM;
wire C_OUT;
// Instantiations:
fulladd4 faordered(SUM, C_OUT, A, B, C_IN);
fulladd4 fa_name(.sum(SUM), .c_out(C_OUT),.a(A),.b(B), .cin(C_IN));
```

By instantiating modules, which may also have modules instantiated inside of them, which may also have modules instantiated inside of them and so on, you can build up a hierarchical design.